Knowledge Distillation and Deployment of a Command-Parsing LM on a Local Cloud

CS3081 Cloud Computing mini project · **Suhas Chowdary** · Roll no. **230101094** · Repository: github.com/Suhas-123-cell/cloud-kd-command-parser (final commit tagged `final`) · Models: Release `models-r2`

> **How to read this report.** Numbers marked MEASURED come from a command whose raw output is saved in `results/` (path given). Statements marked ASSUMPTION are design reasoning or expectations that were not measured. Environment for every measurement: Apple M5 laptop (10 cores, 16 GB), Docker Desktop VM with 10 vCPU / 8 GB, k3d v5 with k3s v1.31.4 (one server node), Ollama 0.40.2 serving `phi3.5` (Q4_0) on the host.

Architecture

Host: Ollama + phi3.5 (teacher). k3d cluster `cc-kd` (from `cluster/k3d.yaml`): namespace `data` runs SeaweedFS (S3 API, PVC-backed); namespace `pipeline` runs a 2-way Indexed Job of teacher workers that read `s3://datasets/commands_pool.jsonl`, call Ollama through `host.docker.internal` and write `s3://datasets/teacher/shard-i.jsonl`. `clean.py` builds `s3://datasets/v1/train.jsonl`; training on the laptop CPU produces `s3://models/v1/` and `/v2/` (+ GitHub Release). Namespace `app` runs the FastAPI parser (Deployment, Service, Traefik Ingress `/parse` on `localhost:8080`, HPA) which downloads its weights from SeaweedFS at start-up with credentials from a Secret.

Task 1 — Object storage and parallel teacher labelling

Implementation. `make up` creates the cluster from `cluster/k3d.yaml`, deploys SeaweedFS (`chrislusf/seaweedfs:3.80`, one `weed server -s3` process) in `data`, generates random S3 keys into the git-ignored `.secrets/` and loads them as Secrets, then creates buckets `datasets` and `models` and uploads `commands_pool.jsonl` (`results/task4/deployment.txt` lists both buckets). The prompt (`prompts/teacher_prompt.txt`) lists the eight intents with short descriptions and the handout's examples, the slot rules, six worked examples (four from the handout, two written for this project, none from `validation.jsonl`) and asks for one JSON line. Ollama options: `temperature 0, seed 42` (fixed by the handout), plus `num_predict 96`, a stop sequence `"\n\n"` (without it phi3.5 appended "Note: …" paragraphs) and **structured output**: `format` is a JSON schema with exactly the four keys and the eight intents (`worker/teacher.py`), so decoding cannot leave the schema; the reply is still phi3.5's own generated text, stored byte-for-byte. Teacher full frame / JSON validity on validation MEASURED `results/teacher_prompt_iterations/`: round 1 first draft 32/81, revised 41/89, + stop 41/99; round 2 + structured output 41/100, then a revised prompt from an error analysis (an invented `"date": "today"` copied from a few-shot example, person names and "near me" extracted as places, indirect weather questions sent to `recommendation_events`) **55/100** (`round2/`, reproduced by `scripts/teacher_prompt_round2.py`). **Disclosed:** both rounds were tuned on `validation.jsonl`, so the teacher's (and hence the student's) validation score is optimistic; a round-2 draft that quoted two phrases occurring in validation was replaced by phrases that do not (score unchanged).

Sharding (≤ half page). Worker `i` of `N` takes the commands whose 0-based line number `n` satisfies `n mod N = i` (`worker/worker.py: select_shard`). The rule is a pure function of the line number, so the shards are disjoint and together cover every line with no coordination, no locks and no shared state; sizes differ by at most one; any worker can be restarted and recomputes exactly the same subset. Round-robin (rather than contiguous blocks) also spreads any ordering in the pool evenly. Each worker writes only its own object `teacher/shard-i.jsonl`, so there are no concurrent writes to one key. The Job uses `completionMode: Indexed` with `completions = parallelism = 2`; `SHARD_INDEX` comes from the Pod's job-completion-index annotation via the downward API, and `SHARD_COUNT=2`. Workers re-upload their object every 50 records and, on restart, skip ids already present, so a killed Pod loses at most 50 replies. Both Pods ran at the same time (started 16:18:56Z, finished after 1,455 s and 1,458 s, about 1.0 command/s each) MEASURED `results/task1/pipeline_job.txt, worker_logs.txt`. The real bottleneck is the single shared Ollama (run with `OLLAMA_NUM_PARALLEL=2` on port 11435 for this run; the manifest default is 11434 and `make label OLLAMA_URL=…` overrides it); more shards would not help without more teacher capacity ASSUMPTION.

Cleaning outcome (`results/clean_counts.json`)	Records
Input (both shards)	3,000
(a) reply is not valid JSON	0
(b) keys not exactly intent/date/time/place	0
(c) intent not one of the eight	0
(d) slot neither null nor text in the command	86
(e) duplicate command (smallest id kept)	50
Kept → `datasets/v1/train.jsonl`	**2,864**
0 + 0 + 0 + 86 + 50 + 2,864 = 3,000 (asserted in `clean.py`)	

Each record is rejected for the *first* rule it breaks, in order (a)–(d); survivors are then de-duplicated on lower-cased, whitespace-normalised text in id order. Rule (d) is a case-insensitive substring test. (a)–(c) are 0 because of the structured output (round 1: 2, 1, 21). (d) fell from 257 to 86; typical rejects are still invented or converted slots, e.g. "read me my calendar for the month" → `"date": "the month of may"` (copied from the prompt's slot examples) or "clear all events from march seven" → `"march 7"`. **Checks** MEASURED: 3,000 ids in the shards, 0 gaps, 0 repeats, every record in the right shard (`results/task1/shard_coverage_check.txt`); `make verify-clean` re-runs `clean.py` on the committed shards and reproduces `4f4a1971…6287` = `datasets/v1/train.sha256` (`results/task1/verify_clean.txt`).

Limitations / improvements. The substring test accepts 16 kept values that match only inside a longer word (e.g. date `"tomorrow"` inside "tomorrows", place `"minneapolis"` inside "minneapoliscity"); a word-boundary match would be stricter. The pool has 50 duplicate texts and all 50 copies passed (a)–(d), hence 50 in row (e). Improvement: run Ollama inside the cluster with its own GPU/CPU budget so workers and teacher scale together.

Task 2 — Training the student (sequence-level distillation)

Implementation. `training/train.py` (plain PyTorch loop + Transformers) fine-tunes `EleutherAI/pythia-70m` on the first 1,500 records of `train.jsonl` (last one C1551; 0 overlap with validation). Each example is `Command: <text>\nJSON: <teacher reply><eos>`; the loss is computed only on the reply tokens. `training/config.yaml` holds seed 42, learning rate 5e-4, batch size 16, 8 epochs, max length 96, warm-up 5 % and the library versions (Python 3.12, torch 2.5.1, transformers 4.46.3); the seed is set in code and `torch.use_deterministic_algorithms(True)`. `make train` reads only that file; `make train SUBSET=50` trains on the first 50 records. v2 uses seed 7 (`make train-v2`). Each model is saved as safetensors + tokenizer + `version.json`, packed into a deterministic tarball (fixed order, mtime 0) whose SHA-256 is line 1 of `models/vN/model.sha256` (other lines: every file). Both are in `s3://models/v1|v2/` and in the Release `models-r2`; the downloaded `model-v1.tar.gz` hashes to `3f1e1bf1…f734`, equal to line 1 MEASURED.

Training loss per epoch, v1 (`training/loss.csv`) MEASURED: 0.772, 0.341, 0.191, 0.114, 0.076, 0.045, 0.024, 0.009 (322 s on CPU; v2: 225 s, final 0.021).

validation.jsonl (100), `tools/score.py`	JSON valid	Intent	Date	Time	Place	Full frame	Raw outputs / report
Teacher phi3.5 (same prompt/settings)	100	84	82	86	94	55	`teacher_validation*.json(l)`
Untrained pythia-70m	0	0	0	0	0	0	`base_pythia70m_validation*`
Student v1 (served)	**100**	85	82	88	88	**56**	`student_v1_validation*`
Student v2 (seed 7)	99	86	81	85	89	56	`student_v2_validation*`

What changed MEASURED. The untrained model continues the prompt with its pre-training habits (YAML lists, GitHub-style JSON with `"homepage": "https://www.sunday.com/"`; shown with the newline stop disabled so its real continuation is visible). After 1,500 examples the student reproduces the teacher's format perfectly and tracks the teacher closely: full frame 56 vs 55, intent 85 vs 84; its output is identical to the teacher's on 64 of 100 commands. Of v1's 44 full-frame misses, 35 are also teacher misses, 9 are its own, and it gets 10 right that the teacher gets wrong, so the teacher is the ceiling. Improving the teacher prompt (Task 1, round 2) raised the student from 39 to 56 full frame with no change to training. It also inherits the teacher's convention errors against SLURP gold. Note: `score.py` counts JSON validity without checking the intent list, but the service does; the round-1 v1 once produced `"intent": "alarm_remove"`, which caused 1 % HTTP 500s under load. The service decoder therefore has a *schema-constrained fallback*: only when the greedy output fails `schema.json` it scores the eight allowed intent prefixes by the student's own log-probability, forces the best one and lets the student finish the slots. Valid outputs are untouched; on validation it fires 0 times for v1 and once for v2 (V049, where v2 repeats "second of january" until the token cap, which a forced intent cannot repair), and the scores above are identical with and without it (`*_greedy.jsonl`).

Sequence-level vs logit-based distillation (≤ one page)

(a) Sequence-level KD (Kim & Rush 2016) trains the student with ordinary cross-entropy on the *sequence the teacher generated* (here its greedy, temperature-0 JSON reply), treating it as if it were the ground-truth label: the student learns the teacher's mode, one hard target per input. **(b) Logit-based (word-level) KD** trains the student to match the teacher's *full next-token probability distribution* at every position, usually by minimising $\mathrm{KL}(p_{\text{teacher},T} \,||\, p_{\text{student},T})$ on temperature-softened logits (often mixed with the hard-label loss), so the student also learns the teacher's "dark knowledge" about

plausible alternatives. **(c) Why it is not possible here.** (1) The teacher is reached only through Ollama's generate API, which returns generated text (a 4-bit GGUF build), not the logit vector over the vocabulary at each step; (2) even with logits, phi3.5 uses a 32k SentencePiece vocabulary and Pythia a 50k GPT-NeoX BPE vocabulary, so the two distributions are over different token sets with different boundaries and cannot be compared position by position without a vocabulary alignment; (3) the handout requires training only on the teacher's generated replies. **(d) A real limitation of what I did:** the student can only be as good as one deterministic teacher reply per command. It copies the teacher's systematic mistakes (full-frame ceiling about the teacher's 55 %), gets no signal about how confident the teacher was, and the cleaning step removes 86 replies where the teacher hallucinated a slot, so commands with no slot information are under-represented in exactly the cases the teacher found hard.

Limitations / improvements. Weights are reproducible with the same seed and versions on the same platform; across CPUs they are not guaranteed bit-identical ASSUMPTION. 8 epochs reach a training loss of 0.009 (likely memorisation; no held-out loss was tracked because validation must not be used for training decisions). Improvements: a held-out split of the teacher data for early stopping; several teacher samples per command; constrained decoding of all four keys rather than only the intent.

Task 3 — The inference service in a container

Implementation. `service/app.py` (FastAPI/uvicorn, port 8000): `POST /parse` returns 200 with only the four-key object and headers `X-Model-Version` (= `MODEL_VERSION`) and `X-Served-By` (Pod name from the downward API); `/healthz` always 200; `/readyz` 503 until the model is loaded. At start-up a background thread downloads every flat object under `MODEL_URI` from `S3_ENDPOINT` (keys from Secret `s3-credentials` via `envFrom`), retries 5 times and then exits so Kubernetes restarts it. `MAX_NEW_TOKENS` caps generation. Every output is validated against `schema.json`; any failure (or exception) gives HTTP 500 `{"error": "invalid_model_output"}`; `FAULT_INJECT=1` replaces the output with `not json` before validation (`results/task3/fault_inject.txt`). Image: `python:3.12.8-slim-bookworm`, CPU-only torch wheel, every package pinned (38 pins from `pip freeze`, then `pip check`), no weights (0 `*.safetensors` files in the image); built with one command, `docker build -t cc-kd/parser:1.0 service/`. Size 1.21 GB on disk, 247 MB compressed MEASURED `results/task3/image_info.txt`. 18 unit tests cover the API contract with a fake model.

Finding: slow matrix multiplies inside the container. At 1 CPU the first image needed ~3.9 s per request. The profiler puts 56 % of the time in `aten::addmm` and 43 % in `aten::mm`: torch's linux/arm64 `F.linear` path (OpenBLAS reports "Core not found, falling back to generic ARMv8" inside the Docker VM) ran the 512→50,304 output layer in 20.0 ms, while a matmul with a *pre-transposed contiguous* weight took 5.4 ms. `pretranspose_linears()` stores W^T once at load time: mean latency on 30 commands fell from 3,713 ms to 683 ms (5.4×) with 30/30 identical outputs MEASURED `results/task3/profiling.txt` (`scripts/profile_linear.py`). On x86_64 with MKL this is probably unnecessary ASSUMPTION.

Results under limits (`scripts/task3_container_test.sh`: `docker run --cpus 1 --memory 1g`, 50 sequential validation commands) MEASURED `results/task3/`: 50 × HTTP 200; latency mean 697 ms, p50 696 ms, p95 786 ms, max 803 ms; **peak memory 995,106,816 B = 949 MiB** (cgroup `memory.peak`), highest `docker stats` sample 607 MiB; `memory.events max 0, oom_kill 0`, `OOMKilled=false`. The gap between the two peaks is page cache from writing and reading the 282 MB weight file during start-up, which the cgroup counts but can reclaim; the peak varies between runs (754 MiB in an earlier run) and leaves little headroom under the 1 GiB limit, so the limit should not be lowered.

Limitations / improvements. One request is generated at a time per replica (a lock; the CPU limit is one core). Batching concurrent requests, int8 weights, or ONNX Runtime would raise throughput; streaming the safetensors file straight into memory would cut the start-up page-cache peak.

Task 4 — Kubernetes deployment, scaling and reliability

Implementation (`k8s/app/parser.yaml`). Namespace `app`; Deployment with 2 replicas, requests 500m CPU / 512 Mi, limits 1 CPU / 1 Gi; readiness probe `/readyz` every 2 s (failureThreshold 1); liveness probe `/healthz` every 10 s after 10 s; RollingUpdate with maxSurge 1 / maxUnavailable 0; a 5 s `preStop` sleep so a terminating Pod keeps serving until it has left the endpoints; ClusterIP Service; Traefik Ingress `/parse`; HPA (autoscaling/v2) min 1, max 2, target 70 % CPU of the request, scale-down stabilisation shortened to 60 s (default 300 s) so the scale-down could be observed. All experiments are scripted in `scripts/experiments.sh`; each saves a 2-second sampler of Deployment/HPA/Pod/endpoint state. Experiments that must *start* from two Ready replicas (cold start, recovery, rolling update, load balancing) raise the HPA floor to 2 for their duration and restore it to 1 afterwards; the HPA experiment uses the specified 1–2.

Experiment (all through the Ingress, 4 Locust users)	Result MEASURED	Evidence in `results/task4/`
Ingress, requests/limits, probes	`curl` through `localhost:8080/parse` → 200, both headers	`deployment.txt`
No ready endpoints while loading	In all three cold-start runs a sample shows endpoints `ready=` (none) `notReady=` both new Pods while the model loads (17:44:51, 17:44:59, 17:45:04); both turn Ready by 17:44:55, 17:45:00 and 17:45:07	`coldstart/state_run{1,2,3}.txt`
HPA 1 → 2 → 1	Idle at 1 replica (17:31:00); load from 17:31:02; 17:31:15 CPU 104 %/70 % and Deployment desired = 2 ("cpu above target"); **2/2 Ready at 17:31:17**; CPU 198–200 % under load; load ended 17:35:02; **back to 1 at 17:36:31** ("all metrics below target"). 1,445 requests, 0 failures	`hpa/hpa_timeline_compact.txt`, `scaling_events.txt`
Rolling update v1→v2 then `rollout undo`, under 5 min load	1,792 requests, **3 failed (0.17 %)**; 1,517 served by v1, 278 by v2; after undo 6/6 probes show `X-Model-Version: v1`. All 3 failures are HTTP 500 (no connection errors); inferred from the offline predictions to be V049, the one command where v2 loops until the token cap (278 v2 requests ≈ 3 passes over the 100 commands). The single probe right after `rollout status` returned v1 from an old Pod still draining during its 5 s `preStop`	`rollout/`
Recovery: delete one Pod under load	Pod deleted 17:42:29; 2/2 Ready again 17:42:35; 1,101 requests, **0 failures**	`recovery/`
Cold start: delete all Pods, ×3	Creation→Ready per run (slowest Pod): 6, 3, 5 s; **median 5 s**. Logs: `downloaded s3://models/v1/model.safetensors (281715176 bytes)`, download 0.7–2.3 s	`coldstart/`
Load balancing, exactly 2 Ready, standard command	742 requests, 0 failures; per replica **374** / **370** (50.3 % / 49.7 %)	`loadbalance/loadtest_headers.json`

Earlier failed attempt. With the round-1 model, the first recovery run (before the fix) had 9/967 HTTP 500s (0.93 %); the pod logs at the time showed all nine were the `alarm_remove` intent, which led to the constrained fallback described in Task 2. Its files were overwritten by the re-run, so this figure is reported from the session, not from a saved file. All runs in the table use the fixed image and the round-2 models.

Limitations. The cluster is a **single k3s node on one laptop**: deleting a Pod shows self-healing, but there is no fault tolerance against a node, disk, Docker VM or laptop failure, and both replicas share the same CPUs. Scaling from 1 to 2 replicas adds no hardware. metrics-server samples every ~15 s, so the HPA reacts in 15–45 s; cold-start times have 1 s resolution and exclude image pulls (the image is pre-imported into the node). SeaweedFS is a single replica on a local-path PVC: losing it loses the models (they can be re-seeded from the Release with `make seed-models`). **Improvements:** multiple agents across machines with Pod anti-affinity, a PodDisruptionBudget, replicated object storage, and a startup probe.

Task 5 — Subnetting and network access control

Subnet	Network	Prefix	Usable range	Broadcast	Hosts
app (1,000)	10.20.0.0	/22	10.20.0.1–10.20.3.254	10.20.3.255	1,022
public (250)	10.20.4.0	/24	10.20.4.1–10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0	/25	10.20.5.1–10.20.5.126	10.20.5.127	126
mgmt (20)	10.20.5.128	/27	10.20.5.129–10.20.5.158	10.20.5.159	30

Dir	#	Rule (public-subnet NACL, `network/nacl.json`)
in	90	deny all from 198.51.100.0/24
in	100/110	allow tcp 80 / 443 from 0.0.0.0/0
in	120	allow tcp 22 from 10.20.5.128/27 (mgmt)
in	130	allow tcp 1024–65535 from 10.20.0.0/22 (app replies)
out	100	allow tcp 8000 to 10.20.0.0/22
out	110	allow tcp 1024–65535 to 0.0.0.0/0 (client replies)

Working in `network/subnets.md` (e.g. app: /23 gives 510 < 1,000, /22 gives $2^{10}-2 = 1{,}022$).

Anything else: default deny. `nacl_sim.py --check` on the sample: 6/6 MEASURED; 140 adversarial packets written by an independent reviewer: 140/140 after correcting one expectation (outbound reply to port 3306, inside the ephemeral range) `results/task5/`.

Why replies need their own rules, and why order matters (≤ half page). A NACL is **stateless**: it does not remember that it let a connection in, so each packet, including the reply to an allowed request, is judged only by the rules for its own direction. A browser's request to port 443 is allowed by inbound rule 110, but the load balancer's reply goes *out* to the client's ephemeral port (1024–65535) and would hit the default deny without outbound rule 110; likewise the app tier's answer from port 8000 comes back *in* to the load balancer's ephemeral port and needs inbound rule 130. (A stateful security group would allow both replies automatically.) **Order:** rules are evaluated from the lowest number and the *first match wins*. A packet from 198.51.100.9 to port 443 matches both "deny 198.51.100.0/24" and "allow tcp 443 from 0.0.0.0/0"; the deny only takes effect because it is numbered 90, lower than 100/110. Numbered 140 it would never be reached for web traffic, and the abusive range could still use ports 80/443 (sample packet S04 checks this). **Trade-off:** with port-only matching, outbound rule 110 also lets any tcp packet to a port ≥ 1024 leave (e.g. to port 8000 outside the app subnet); NACLs cannot see source ports or connection state, so this is the usual price of statelessness and is why security groups are layered on top.

Task 6 — Reproducibility, testing and CI/CD

Reproducibility. Everything except secrets and weights is in Git: code, kit, prompt, raw shards, cleaned data, configs, manifests, results. `make up` builds and deploys the whole system from a fresh clone in three commands (`git clone`, `cd`, `make up`); it downloads both model versions from the Release `models-r2` and verifies them against the committed checksums. Verified by deleting the cluster and running exactly those commands in an empty directory: exit code 0 after 206 s, ending with "/parse answers through the Ingress" MEASURED `results/task6/make_up_fresh_clone.txt`; `make down` deletes the cluster (`results/task6/make_down.txt`).

CI (`.github/workflows/ci.yml`), in order via `needs`: **lint** (ruff) → **unit-tests** (18 pytest tests) → **model-evaluation** (downloads `model-v1.tar.gz` from the Release with `gh release download`, checks its SHA-256 against `models/v1/model.sha256`, runs `predict.py` + `score.py` on `validation.jsonl`, uploads the outputs) → **quality-gate** (`score.py --min-validity 95`). A parallel **secret-scan** job runs gitleaks over the full history. A `workflow_dispatch` input `model=untrained` evaluates the base pythia-70m instead.

- Run that **fails at the quality gate** (untrained Pythia-70M, JSON validity 0 < 95): actions/runs/42504978056
- Passing run on the trained model (all five jobs green): actions/runs/42504548091.
- Secret scanning: gitleaks reports no leaks. One false positive was allow-listed in `.gitleaksignore`: saved `docker history` output contained `ENV GPG_KEY=…` from the python base image, the public fingerprint of the CPython release-signing key (now redacted in the file).
- `AI_USE.md` lists the AI tools used, what for, and how each output was checked.

Limitations / improvements. CI does not build the image or deploy to an ephemeral k3d cluster; adding a job that runs `make up` + a smoke test in the runner would cover Tasks 3–4 too. The untrained-model run depends on the Hugging Face Hub being reachable.

Task 7 — Performance, cost and review

Standard load test (exact marking command, 2 Ready replicas, through the Ingress; `results/loadtest_stats.csv`, `loadtest_output.txt`, `loadtest_headers.json`, `kubectl_top_during_loadtest.txt`) MEASURED:

p50	p95	p99	Throughput	Error rate	CPU per Pod (`kubectl top`, every 10 s)	Memory per Pod	Cold start
650 ms	1,000 ms	1,200 ms	6.24 req/s	0 / 744 = 0 %	932–1,001m in every sample once metrics show the load (limit 1000m)	500–514 Mi (limit 1 Gi)	5 s median (Task 4)

All request counts in this report are from `*_stats.csv`, which Locust writes just before shutdown; its console summary and the header counter in the locustfile include the last few in-flight requests (standard run 748, rollout 1,795, recovery 1,103, load balancing 744). No pass/fail result changes. Both replicas were CPU-saturated, so throughput is CPU-bound: about 3 req/s per core.

Cost (`data/rate_card.json`: 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; billed on limits for every running replica): per replica 1 vCPU × 3.50 + 1 GiB × 0.45 = **3.95 INR/h**; two replicas = **7.90 INR/h**. Requests per hour = 6.2412 req/s × 3,600 = **22,468**. Cost per 1,000 requests = 7.90 / 22,468 × 1,000 = **0.352 INR**. ASSUMPTION: the measured throughput is sustained for the whole hour and both replicas run the whole hour (with the HPA idle at 1 replica the hourly cost would fall to 3.95 INR/h, but so would capacity).

SLO. "At least 99 % of `/parse` requests return HTTP 200 within 30 s", window = one standard load-test run. Error budget = 1 % × 744 = 7.44 → **7 failed requests**. Used: **0 of 7 (0 %)**. For comparison the rolling-update run used 3 of its 17-request budget (1 % of 1,792).

Concern	Evidence (measured)	Limitation found	Improvement
Data and storage	3,000 → 2,864 kept; 86 (2.9 %) rejected for invented slots (round 1: 257); structured output removed (a)–(c); train.jsonl checksum reproduced	Teacher still invents or converts some slots; substring rule accepts 16 in-word matches; one SeaweedFS replica	Word-boundary slot check; replicated storage
Model quality	Student full frame 56 % vs teacher 55 % (round 1: 39 vs 41); validity 100 %; 35 of 44 misses shared with the teacher	Teacher is the ceiling; prompt tuned on validation, so scores are optimistic; v2 loops on one command	A separate labelled dev set for prompt work; repetition penalty or constrained slot decoding; several teacher samples
Latency and throughput	p50 650 ms, p95 1,000 ms, 6.24 req/s on 2 replicas	One request at a time per replica; arm64 BLAS path 5× slower until pre-transposed	Request batching, int8 weights or ONNX Runtime, KV-cache reuse of the shared prompt prefix
Resource utilisation	~1 core per Pod at limit; 500–514 Mi of 1 Gi in the cluster; container cgroup peak 949 MiB (incl. page cache)	CPU-bound; request 500m makes HPA read 200 % under load; little memory headroom at start-up	Set requests closer to real use (~1 core); keep the 1 Gi limit and stream the weights to cut the start-up peak
Reliability and recovery	Recovery 0/1,101 failed, back to 2 Ready in 6 s; rollout 0.17 % failed; cold start 5 s	Single node, single storage replica; no PDB	Multi-node cluster with anti-affinity, PodDisruptionBudget, startup probe
Security and cost	gitleaks: 0 findings; random S3 keys only in Secrets; 0.352 INR / 1,000 requests	One admin S3 identity shared by all namespaces; no TLS; billed for idle 2nd replica	Per-namespace least-privilege keys, TLS at the Ingress, NetworkPolicies, scale-to-1 off-peak

References

Kim & Rush, Sequence-Level Knowledge Distillation, EMNLP 2016. Hinton, Vinyals & Dean, Distilling the Knowledge in a Neural Network, 2015. Chen et al., Distilling Knowledge Learned in BERT for Text Generation, ACL 2020. Biderman et al., Pythia, ICML 2023. Bastianelli et al., SLURP, EMNLP 2020. Kubernetes, k3d, SeaweedFS, Traefik, Locust, Hugging Face Transformers and TRL documentation; AWS VPC Network ACLs; Google SRE Book, ch. 4 (SLOs).